Campus Event Coordination System (CECS) Requirements Gathering & Modeling Document

1. Project Leader – Timilsena, Kamal Raj (hdd5ps)

2. Requirements and Design Lead – Bryington, Mason Toby (xur6rh)

3. Implementation and Integration Lead - Muwanguzi, Precious Solomon (max7jn)

Part 1 — Project Scope & Stakeholders

1.1 System Vision

Problem Statement

Campus events are currently advertised across fragmented channels (emails, posters, social media, informal messages), causing low visibility, missed opportunities, and scheduling/resource conflicts. Cultural-credit check-in processes can be confusing or unreliable, forcing manual recovery by hosts. Students also lack a unified way to locate events via clear campus mapping.

Business Objectives

Centralize event discovery and event information to increase student awareness and participation.
Improve reliability and traceability of cultural-credit eligibility and attendance tracking.
Reduce event scheduling conflicts and improve coordination via structured event lifecycle and approvals.
Support navigation to events using a campus map and external directions links.

System Boundaries

Included (In Scope)

- User login (prototype may use mock institutional login) and role assignment.
- Event browsing (chronological feed), search/filter by tags/time/location/cost/credit eligibility.
- Event creation/editing by staff/club officers with Draft/Pending/Approved/Rejected/Archived lifecycle.
- Approval workflow for administrators (approve/reject).
- Cultural credit designation visibility and attendance check-in with timestamp and status.
- Embedded campus map pin and external directions link.
- Save filtering preferences and reminders/notifications (app-level).

Excluded (Out of Scope)

- Payment processing / ticket transactions.
- Mandatory hardware-based check-in as a requirement (device features may vary; fallback required).
- Enterprise analytics dashboards, external calendar integrations, SMS/push notification infrastructure in prototype.

1.2 Stakeholder Identification

Student (Primary User)

Goals:

- Discover events aligned with interests; view cultural credit eligibility; check in reliably; find locations quickly.

Primary concerns:

- Easy navigation; accurate info; privacy of attendance data.

Club Officer / Staff Host (Primary User)

Goals:

- Create/edit events; publish accurate info; tag events properly; manage drafts.

Primary concerns:

- Approval delays; clarity of required fields; editing flexibility.

Campus Administrator (Organizational Stakeholder)

Goals:

- Ensure compliance and quality; approve/reject events.

Primary concerns:

- Workflow efficiency; preventing duplicates/inappropriate postings.

Student Affairs (Organizational Stakeholder)

Goals:

- Audit cultural credit fulfillment; verify eligibility and timestamps.

Primary concerns:

- Data accuracy and traceability.

Facilities Staff (Organizational Stakeholder)

Goals:

- Avoid room/resource double-booking; maintain operational clarity.

Primary concerns:

- Last-minute changes; scheduling conflicts.

University IT / IT Administrator (Technical Stakeholder)

Goals:

- Maintain uptime and data integrity; secure access control.

Primary concerns:

- Security vulnerabilities; reliable persistence.

Campus Security (Organizational Stakeholder)

Goals:

- Visibility into high-attendance events for safety planning.

Primary concerns:

- Timely info on major gatherings.

Part 2 — Requirements Specification

2.1 Functional Requirements

FR-01: The system shall authenticate users using institutional login for the prototype (mock credentials acceptable).

FR-02: The system shall assign exactly one role to each authenticated user (Student, Staff, Administrator).

FR-03: The system shall display a chronological feed of approved upcoming events.

FR-04: The system shall allow users to view an event detail page including title, host, date/time, location, cost, description, tags, and cultural credit status.

FR-05: The system shall allow users to filter events by tags (e.g., “Free Food”, “Free Entry”).

FR-06: The system shall allow users to filter events by time and location.

FR-07: The system shall allow users to filter events by cost (Free vs Any).

FR-08: The system shall allow users to filter events by cultural credit eligibility (Eligible vs Any).

FR-09: The system shall allow staff users to create events including title, date/time, location, description, cost, and cultural credit designation.

FR-10: The system shall allow staff users to save events as Draft and submit events for approval (status becomes Pending Approval).

FR-11: The system shall allow administrators to approve or reject pending events.

FR-12: The system shall prevent events that are not approved from appearing in the public event feed.

FR-13: The system shall record a timestamped attendance check-in for a student at an event and mark status as “Checked in”.

FR-14: The system shall provide a fallback check-in option (e.g., event code entry) when primary check-in fails or is unsupported.

FR-15: The system shall display event location using an embedded map interface with a pin at the event location.

FR-16: The system shall provide a link to open directions in an external mapping application.

FR-17: The system shall allow users to save event filtering preferences and apply them on future sessions.

FR-18: The system shall allow users to enable event reminders for saved/upcoming events.

2.2 Nonfunctional Requirements

Performance

NFR-01: The event feed shall load within 2 seconds under 500 concurrent users.

NFR-02: Filtering operations shall return updated results within 1 second.

Security

NFR-03: User credentials shall be stored using salted cryptographic hashing.

NFR-04: Administrative actions (approve/reject, publish control) shall require role-based authorization checks on every request.

Reliability / Availability

NFR-05: Approved event and attendance data shall persist after a system restart (verified by restart-and-reload test).

NFR-06: The system shall maintain 99% availability during standard academic hours (excluding scheduled maintenance).

Usability

NFR-07: A first-time user shall navigate from login to viewing an event detail page within ≤ 3 interactions.

Maintainability

NFR-08: The system shall separate business logic from presentation logic to support future extensibility (verified by code/module review rubric).

Portability

NFR-09: The application shall render correctly on modern mobile browsers (iOS Safari and Android Chrome) without critical layout breakage.

2.3 Requirement Conflicts

Conflict 1: Low-friction login vs security strength

Why it exists: Stronger authentication increases security but reduces ease of access.

Resolution strategy: Prototype uses mock/institution-style login; defer advanced MFA requirements, maintain role checks.

Conflict 2: Rich filtering vs performance

Why it exists: Many filter combinations can increase query complexity and slow response.

Resolution strategy: Limit combinations if needed and optimize indexing/caching strategies later (tracked as a requirement risk).

Conflict 3: Embedded mapping vs scope control (and mobile usability)

Why it exists: Full navigation integration adds complexity and can clutter small screens.

Resolution strategy: Use simple embedded pin view + external directions link; avoid full routing APIs in prototype.

Conflict 4: Staff free-text tags vs accurate student filtering

Why it exists: Free-text tags reduce filter consistency, hurting discovery accuracy.

Resolution strategy: Require tags chosen from a controlled list for filtering.

Part 3 — Requirements Modeling

3.1 Scenario-Based Model (Use Cases)

UC-01: Browse & Filter Events

Primary Actor: Student

Preconditions: User is authenticated (prototype login acceptable).

Main Flow:

1. Student opens event feed.
2. System displays chronological approved events.
3. Student selects filters (tags/time/location/cost/credit).
4. System displays filtered results.

Alternative Flow: Student clears filters to return to full feed.

Exception Flow: No matches → system shows “No results” and suggests clearing filters.

UC-02: Create Event (Draft → Submit for Approval)

Primary Actor: Staff/Club Officer

Preconditions: User has Staff role.

Main Flow:

- Staff selects “Create Event.”
- Staff enters required event fields.
- Staff submits event.
- System sets status to Pending Approval.

Alternative Flow: Staff saves as Draft instead of submitting.

Exception Flow: Missing required fields → system blocks submit and shows validation messages.

UC-03: Approve or Reject Event

Primary Actor: Administrator

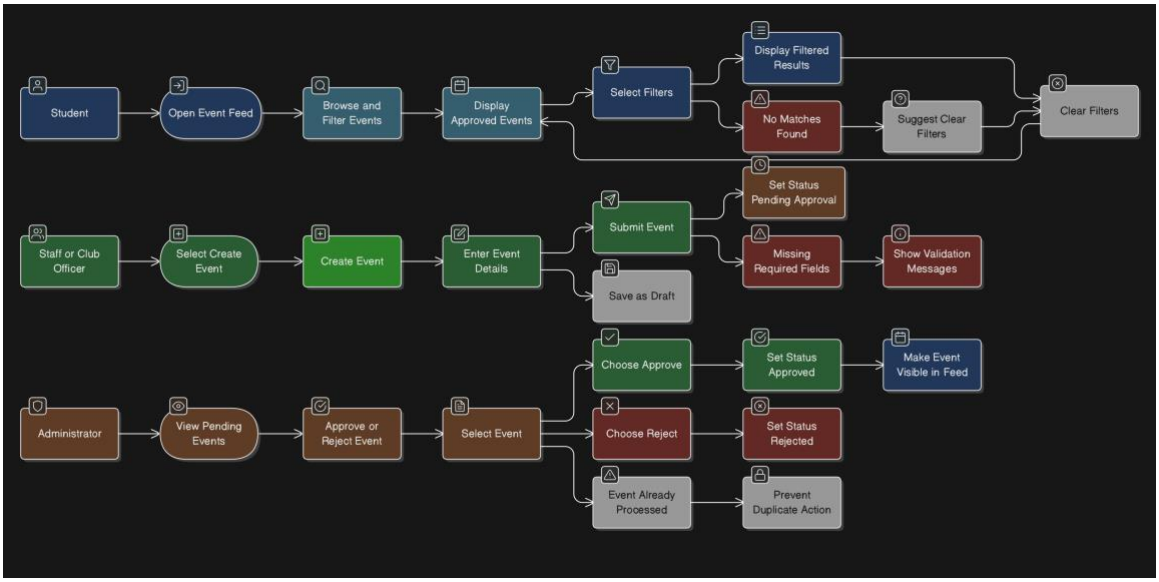
Preconditions: User has Administrator role; event is Pending Approval.

Main Flow:

- Admin views pending events.
- Admin selects an event and chooses Approve.
- System changes status to Approved and makes it visible in feed.

Alternative Flow: Admin chooses Reject → status becomes Rejected.

Exception Flow: Event already processed → system prevents duplicate action.



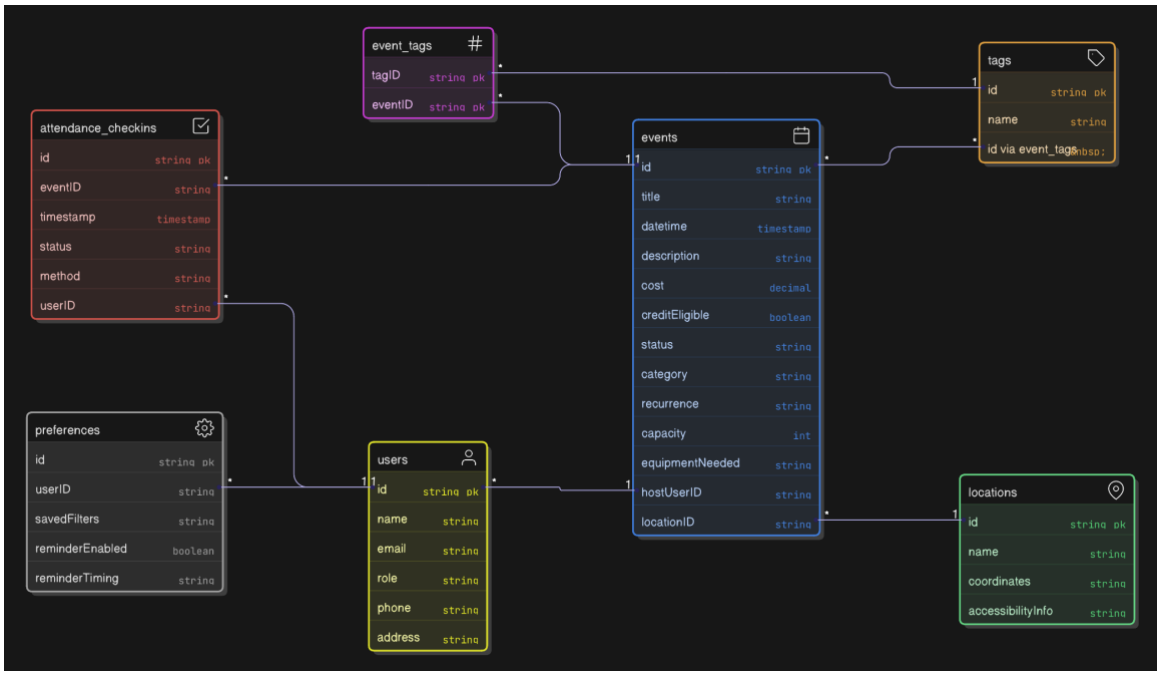
3.2 Data Model (ERD)

Entities (final set):

- User(UserID, name, email, role)
- Event(EventID, title, datetime, description, cost, creditEligible, status, hostUserID, locationID)
- Location(LocationID, name, coordinates)
- Tag(TagID, name)
- EventTag(EventID, TagID) — join for many-to-many
- AttendanceCheckIn(CheckInID, EventID, UserID, timestamp, status, method)
- Preference(PreferenceID, UserID, savedFilters, reminderEnabled, reminderTiming)

Key relationships / cardinalities:

- User (1) — (Many) Event (host/creator)
- Event (Many) — (1) Location
- Event (Many) — (Many) Tag (via EventTag)
- User (1) — (Many) AttendanceCheckIn; Event (1) — (Many) AttendanceCheckIn



3.3 Class Model

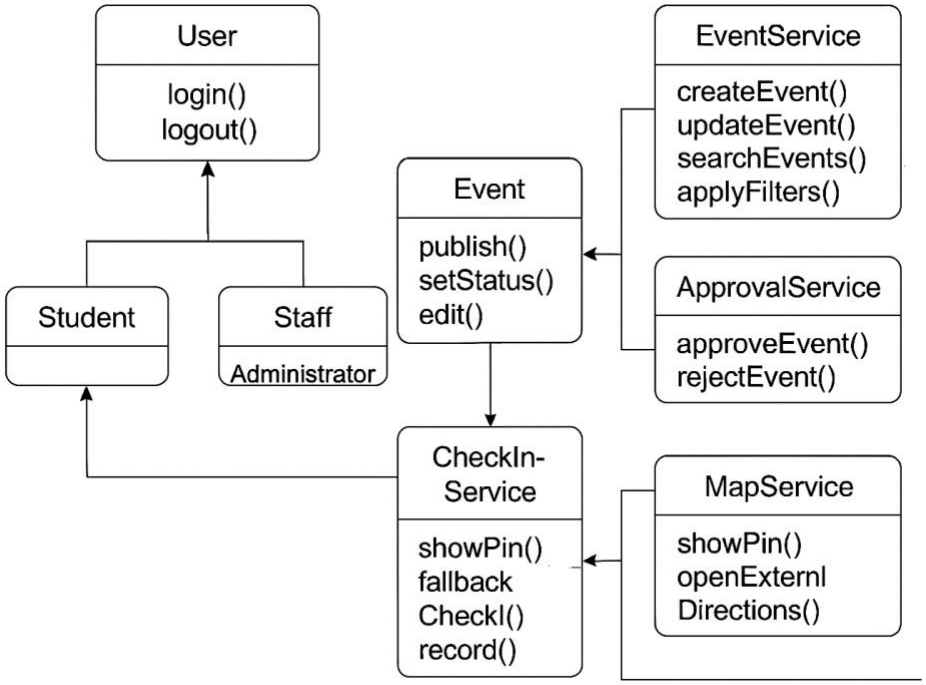
Classes (Object Oriented responsibilities):

- User (login(), logout()) with subclasses Student, Staff, Administrator
- Event (publish(), setStatus(), edit())

- EventService (createEvent(), updateEvent(), searchEvents(), applyFilters())
- ApprovalService (approveEvent(), rejectEvent())
- CheckInService (checkIn(), fallbackCheckIn(), recordTimestamp())
- MapService (showPin(), openExternalDirections())

How it differs from the Data Model

The data model describes persistent storage structures and relationships (entities + cardinalities). The class model describes runtime objects, inheritance, and high-level behaviors/methods that implement the requirements.



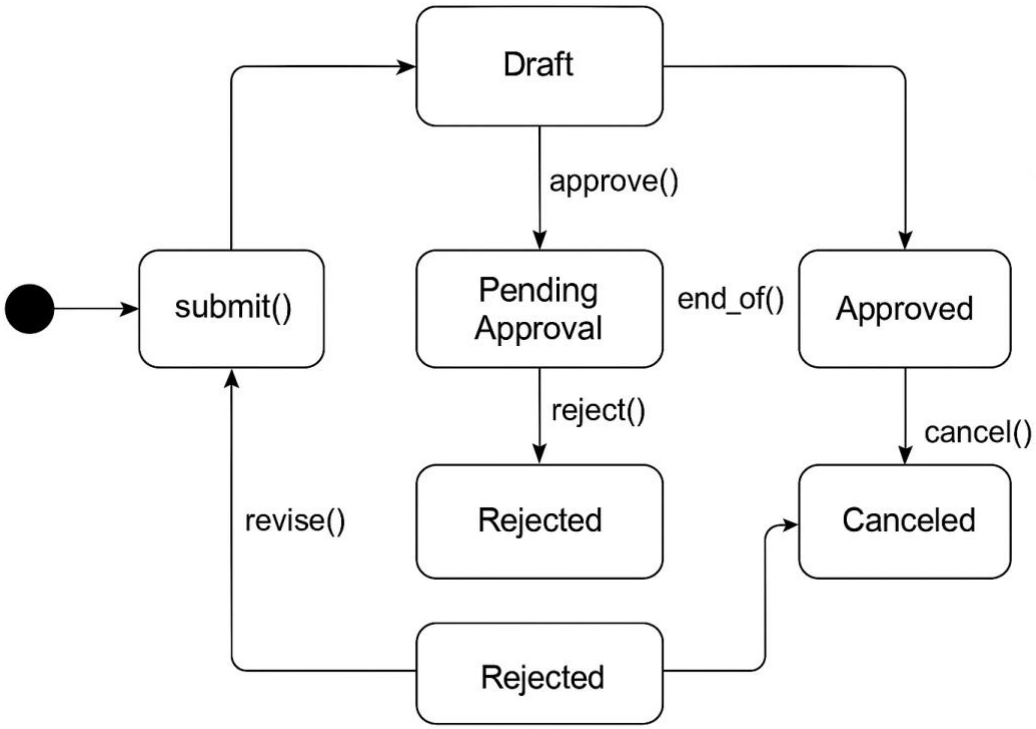
3.4 Behavioral Model (State Diagram for Event)

States: Draft → Pending Approval → Approved → Archived; plus Rejected and Canceled as applicable.

Sample transitions:

- Draft → PendingApproval on submit()
- PendingApproval → Approved on approve()
- PendingApproval → Rejected on reject()
- Approved → Archived on end_of_event()
- Approved → Canceled on cancel()

- Rejected → Draft on revise()



Part 4 — Traceability Matrix

The table below demonstrates traceability by linking requirements to models and validation approaches.

Requirement ID	Modeled In	Validated By
FR-01	UC-01 (precondition), Class (User)	Login test + role assignment test
FR-02	Use Case Diagram, Class Model	Role-based access tests
FR-03	UC-01, Data Model (Event.status)	Feed shows only Approved events
FR-04	UC-01/Details, Data Model (Event)	Event detail page field verification

FR-05	UC-01, Data Model (Tag/EventTag)	Filter correctness tests
FR-06	UC-01, Data Model (Location)	Time/location filter tests
FR-09	UC-02, Class (EventService)	Create event scenario tests
FR-10	UC-02, State Diagram	Draft→Pending transition tests
FR-11	UC-03, State Diagram	Approve/reject workflow tests
FR-12	UC-03, State Diagram	Visibility validation test
FR-13	Check-in interaction, Data Model (AttendanceCheckIn)	Timestamp persistence tests
NFR-01	Architecture (performance)	Load testing (500 concurrent)
NFR-03	Security review + Class Model	Credential storage verification

Part 5 — Validation & Reflection

Reflection Questions

1) Which requirement was hardest to define? Why?

Event visibility constraints (only Approved events appear) were hardest because they require precise lifecycle/state logic and clear rules to prevent premature publication.

2) Did modeling reveal missing requirements? Provide examples.

Yes. Behavioral modeling revealed the need for a Draft state and explicit transitions before submission/approval, and use-case exception flows revealed the need for fallback paths when primary check-in is unsupported or fails.

3) Which nonfunctional requirement most influenced system structure?

Maintainability (separating business logic from presentation) strongly influenced the architecture and class responsibilities.

4) If this project were real, what requirement risks concern you most?

Requirement volatility is the biggest risk, especially changes to cultural credit policies or authentication/compliance rules that could force data model and workflow changes.